

4.5 Arduino 实时音频处理实验

一、实验目的

- 1、学习掌握用 Arduino UNO 进行实时音频信号处理；
- 2、学习掌握 PCB 电路板的设计和制作；
- 3、学习掌握 Arduino UNO 音频处理扩展板的设计与制作。

二、实验设计方案

数字信号处理，并不是数字信号处理器（DSP）独占的领域。Arduino 具有进行实时音频信号的处理的能力。

1、可调正弦波信号源

Arduino 能产生的不仅数字信号，而且是高质量的波形。输出信号是连续的正弦波，可在较宽的频率范围内调整。压控振荡器（VCO）是电子合成器中最重要元件之一。它被用来创建基本的音调，并由滤波器，混频器和包络变成了电子合成声。

可通过调节电压来改变信号源的频率。使用一个 10k 的电位器，使 UNO 板的 A0 的输入电压改变，从而改变输出信号的频率。得到的正弦波发生器，可以在许多音频领域中应用。

在下面示例程序中，包含两个函数。

第一，void calcsine()，计算一个正弦波表。一系列正弦值不是外部预计算的，而是由单片机本身的一次计算得到。

第二，ISR (TIMER2_OVF_vect) 函数每 16 μ s 被调用，以频率 62.5kHz 自动发生中断。中断函数第四次调用时，获取的 A0 模拟输入值被用于 loop() 中正弦波频率的设置。

在 setup()，将 Timer2 设置为快速 PWM 模式，而中断是在上述时间间隔触发。这些通过直接访问 ATmega328 的定时器寄存器来设置。

压控振荡器程序示例如下：

```
// Voltage-Controlled Oscillator
#define sbi(PORT,bit) (PORT |=1<<bit)
#define cbi(PORT,bit) (PORT &=~(1<<bit))
const int pwmOutPin = 11; //PWM output
const int testPin = 3;
const float pi = 3.141592;
int indexCnt, interCnt; //index, interrupt counters
byte sine[512]; //array for sine values
// interrupt control variable
volatile boolean adcSample;
volatile byte bufferAdc0;

void setup () {
  pinMode(pwmOutPin, OUTPUT); // sets the PWM pin as output
  pinMode(testPin, OUTPUT); // sets the test pin as output
  calcSine(); // load memory with sine table
  // set adc prescaler to 64, 8-Bit ADC and VCC Reference
  // select analog channel 0
  ADCSRA |= 0b00000110;
  ADMUX |= 0b01100000;
```

```

// set Timer2 to fast PWM mode
TCCR2A |= 0b10000011;
// Timer2 no prescaler
TCCR2B |= 0b00000001;
cbi (TCCR2B, CS21) ;
cbi (TCCR2B, CS22) ;
// disable Timer0, enable Timer2 interrupt
cbi(TIMSK0,TOIE0);
sbi(TIMSK2,TOIE2);
}
void loop () {
    while (!adcSample) { } // wait for new ADC samples
    digitalWrite (testPin, HIGH) ;
    adcSample = false;
    OCR2A = sine[indexCnt]; // send current value to PWM
    indexCnt++;
    indexCnt += bufferAdc0;
    indexCnt &= 511; // constraint counter index to between 0 and 511
    digitalWrite (testPin, LOW);
}

// calculate sine array with 512 values between 0 and 2 * pi
void calcSine () {
    for (int n =0; n <= 511; n++) {
        sine [n] = 127 * sin (pi * n / 255) + 128;
    }
}

// get potentiometer value
ISR (TIMER2_OVF_vect) {
    interCnt++; // reduce sampling rate 62.5 kHz through 15.625 kHz
    if (4 == interCnt) {
        bufferAdc0 = ADCH; // get 8-bit ADC value
        adcSample = true;
        sbi (ADCSRA, ADSC) ; // start next conversion
        interCnt = 0;
    }
}

```

2、数字信号处理

除了产生高质量的模拟信号，Arduino 也可以作为一个信号处理器。在这一部分中，我们以混响发生器为例来说明。

以模拟技术产生人工混响一直赋有挑战性。高、低和带通滤波器的制作相对简单，但产

生混响是困难的。先是采用诸如混响板或弹簧的机械装置进行尝试，后来，采用组桶式存储器（bucket brigade storage）。

在微控制器中，混响效果可以使用循环缓冲存储器实现。为此，模拟信号首先像往常一样通过 ADC 数字化。Arduino 的 ADC 工作在输入电压 0 和 5V 之间，所以一个 2.5 V 的偏置电压必须被添加到输入信号上。通过耦合电容，将调整后在 0 至 5V 的范围内的信号发送到 ADC 输入端。图 4.16 为用于此目的的简单电路。

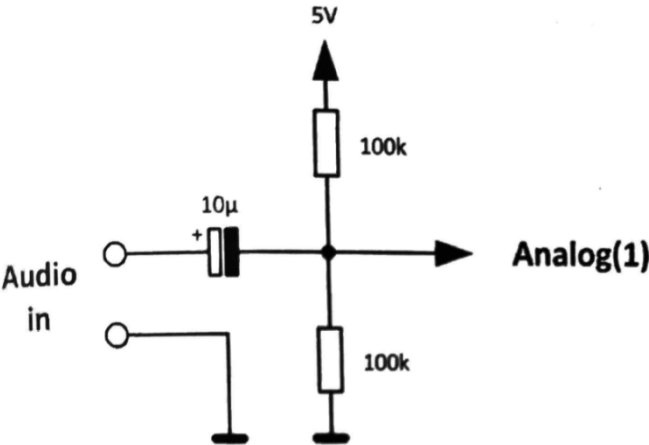


图 4.22 混响发生器输入电路

数字化值存储在数组中。为了产生的混响效果所需的延迟，反复读取循环缓冲区。延时用一个单独的电位器调节。

由于 ATmega328 的资源有限，我们限制数组为 512 字节。

在环形缓冲区，这个存储器阵列构成一个循环中，一个指针用于访问写入数据的内存位置和第二个指针来访问读取位置。完成一个步骤，两个指针都指向下一个位置。如果指针到达数组的结尾（511），指针又被设置为 0，旧的数据将被覆盖。读写指针之间的距离决定了时间延迟的长度。

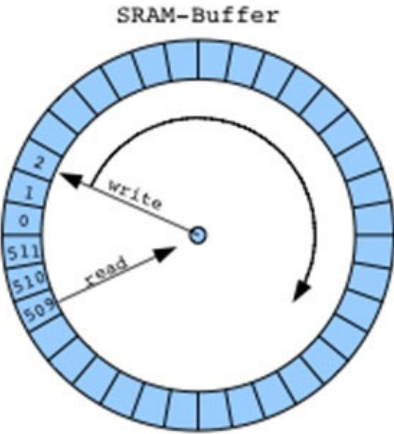


图 4.23 环形缓冲区

在微控制器中，混响效果可以使用循环缓冲存储器实现。数字化值存储在数组中。为了产生的混响效果所需的延迟，反复读取循环缓冲区。延时用一个单独的电位器调节。

与前面在正弦波发生器相同，使用快速 PWM 模式，在 `setup()` 中的相关的设置也相同。

这里，电位器值和模拟输入信号都由中断程序采样。为此，交替地读出 A1（模拟输入）和 A0（电位器）。

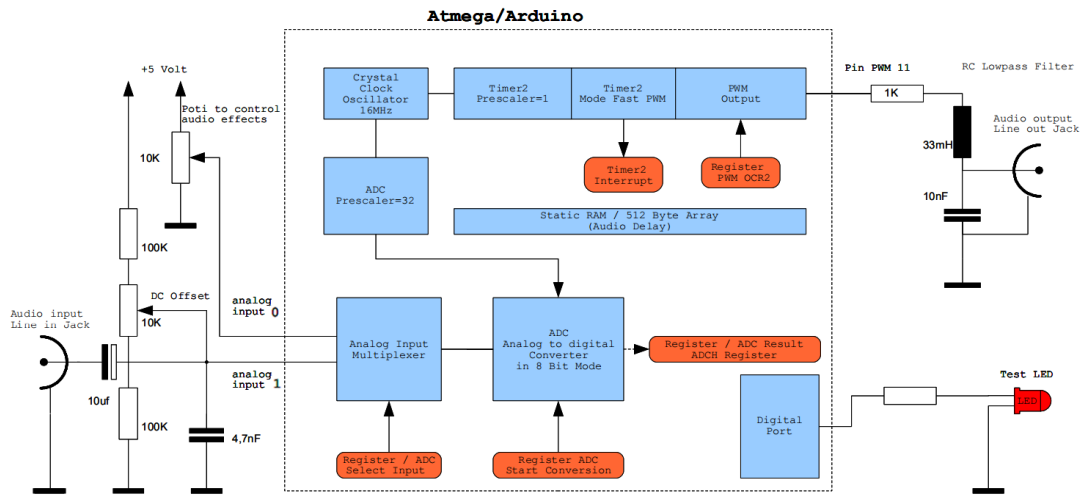


图 4.24

在主程序中，循环缓冲器被写入并在延迟后读取，时间延迟与 A0 处的电位器电压成比例。

混响发生器示例程序

```
// Reverb Generator
#define sbi(PORT,bit) (PORT |=1<<bit)
#define cbi(PORT,bit) (PORT &=~(1<<bit))
const int pwmOutPin = 11; // PWM output
const float pi = 3.1415926535;
const byte halfMax = 127;
boolean div2, selChan; // control variables for sampling
// interrupt variables accessed globally
volatile boolean adcSample;
volatile byte potSampler, analogSample;
int indexCounter, valAnalog, valPot;
byte pwmBuf, ringbuff [512] ; // Audio Ringbuffer Array 8-Bit

void setup () {
    pinMode (pwmOutPin, OUTPUT) ; // set the PWM pin as output
    // adjust adc prescaler
    cbi (ADCSRA, ADPS2) ; sbi (ADCSRA, ADPS1) ; sbi (ADCSRA, ADPS0) ;
    sbi (ADMUX, ADLAR) ; sbi (ADMUX, REFS0) ; cbi (ADMUX, REFS1) ;

    // Set muxer ADC0
    cbi (ADMUX, MUX0) ; cbi (ADMUX, MUX1) ; cbi (ADMUX, MUX2) ; cbi (ADMUX,
    MUX3) ;

    // fast PWM for Timer2
    cbi (TCCR2A, COM2A0) ; sbi (TCCR2A, COM2A1) ;
```

```

sbi (TCCR2A, WGM20); sbi (TCCR2A, WGM21) ; cbi (TCCR2B, WGM22) ;
// set Timer2 Prescaler
sbi (TCCR2B, CS20) ; cbi(TCCR2B, CS21); cbi (TCCR2B, CS22) ;
// Timer2 PWM Port Enable
sbi (DDRB, 3) ; // set digital pin 11 to output
//cli () ; // disable interrupts to avoid distortion
cbi (TIMSK0,TOIE0) ; // disable Timer0 !!! delay is off now
sbi (TIMSK2,TOIE2) ; // enable Timer2 Interrupt
valAnalog = analogSample;
}
void loop ( ) {
while (! adcSample) { }
adcSample = false;
// scale delayed sample with pot value
valPot = (halfMax - ringbuff[indexCounter]) * potSampler/255;
valAnalog = halfMax - analogSample;
valAnalog = valAnalog + valPot;

// limit analog value (min/max)
if (valAnalog < -halfMax) valAnalog = -halfMax;
if (valAnalog > halfMax) valAnalog = halfMax;

// add offset to PWM buffer
pwmBuf = halfMax + valAnalog;
ringbuff [indexCounter] = pwmBuf;
indexCounter++;

// reduce buffer index to 0...511 intervall
indexCounter = indexCounter & 511;

OCR2A = pwmBuf; // write current sample to PWM channel
}
// get analog and pot value
ISR (TIMER2_OVF_vect) {
PORTB = PORTB | 1;
div2 =! div2; // reduce sampling frequency by factor 2
if (div2) {
selChan = ! selChan;
if (selChan) {
potSampler = ADCH;
sbi (ADMUX, MUX0); // set muxer to ADC1
}
}
else {
analogSample = ADCH;
}
}

```

```

    cbi (ADMUX, MUX0); // set muxer to ADC0
    adcSample = true;
  }
  sbi (ADCSRA, ADSC); // start next conversion
}
}

```

3、音频输出放大器

使用扬声器可以获得较好的音频质量。问题是，其中一个是阻抗仅仅几欧姆，因此不能直接连接到 `aduino` 板的引脚，为此需要一个放大器，LM386 一个非常合适我们应用的放大器，它只需要几个额外元件构成电路，如图 3 所示。除了放大器，还具有音频滤波器的功能。2.2 μ F 电容器负责放大器的直流去耦，并且具有一定的高通特性，而 1 μ F 电容器提供低通滤波。当使用简单的脉宽调制产生音频信号时，高频 PWM 谐波将被很好地抑制，而在较低的千赫范围内所需的音频频率传递到放大器。

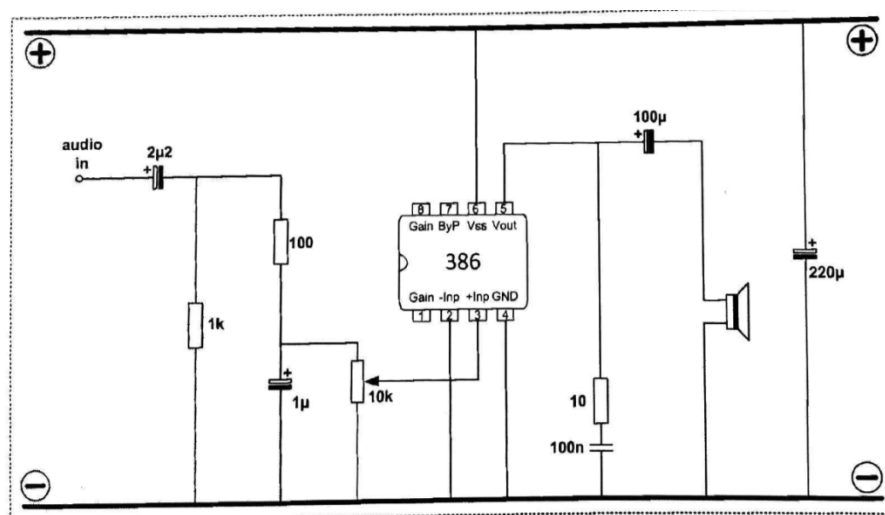


图 4.25 基于 LM386 音频放大器电路图

三、实验任务与要求

- 1、用 Arduino UNO 实现音频信号处理设计，在 1) 音频正弦波信号源 2) 混响发生器中选择一个电路，并分别加上 3) 音频输出放大器，形成一个完整的电路，完成相应器件的选择，制作 Arduino UNO 扩展板。
- 3、编制与调试相关程序。
- 4、将制作的扩展板与 Arduino UNO 板组装后，进行系统联调。